

HOTEL RESERVATION AND MANAGEMENT SYSTEM

PROJECT REPORT

1. Introduction and Problem Statement

The Hotel Reservation and Management System is a Java desktop application created to combine room reservation, guest information, billing, and reporting in one system. The project focuses on practical problems that can occur in hotel front-desk operations, including duplicate reservations during simultaneous requests, disconnected billing processes, loss of information after failures, and limitations of inflexible interfaces. The proposed system addresses these issues through a graphical interface, structured business logic, database persistence, and backup support.

2. Aim and Objectives

The aim is to design and evaluate a thread-safe hotel management application using Java Swing, object-oriented programming, JDBC, Java object serialization, and explicit synchronization. Its objectives are to validate reservation details, prevent overlapping confirmed bookings, calculate charges including additional services, maintain reservation records, generate invoices, and provide a recovery mechanism through serialized backup data.

3. System Architecture and Technologies

The implementation is organized into five main areas: custom domain exceptions, domain entities, data access, business logic with concurrency control, and the presentation layer. Person, Guest, Room, Reservation, and ServiceItem represent the main domain objects. DatabaseManager handles JDBC-based database operations, HotelManager contains reservation and billing rules, and HotelSystemApp provides the Swing graphical user interface. The project also uses serialization to create a backup file.

4. Booking Algorithm

The booking procedure validates the request before changing any reservation data. A shared synchronization lock is used around the critical booking section so that simultaneous requests cannot both confirm the same room and dates.

- Input capture: collect guest name, email, phone, room number, check-in date, and check-out date.
- Validate input: reject blank values and ensure the check-out date is later than the check-in date.
- Acquire bookingLock before checking or modifying shared reservation data.
- Verify that the requested room exists in the inventory.
- Compare the requested dates with confirmed reservations for the same room.
- Use the overlap rule: new check-in < existing check-out AND new check-out > existing check-in.
- If an overlap is found, raise RoomUnavailableException and reject the booking.
- If no conflict exists, save the guest and reservation and assign CONFIRMED status.
- Release bookingLock after the operation and return the confirmation.

5. Billing Algorithm

- Calculate the stay duration in nights using the difference between check-in and check-out.
- Calculate room cost = number of nights × price per night.
- Add the costs of selected ancillary services to obtain the subtotal.
- Apply the specified percentage discount to the subtotal.
- Apply the specified tax to the discounted amount to obtain the final bill.
- Write the invoice to a text file, log the payment, and update the reservation status.

6. Testing and Validation

The documented tests cover invalid dates, normal booking, conflict prevention, concurrent booking attempts, ancillary billing, and backup recovery. Boundary checks reject invalid or zero-length stays, while blank guest fields are rejected before database queries. A multithreaded simulation sends two booking requests for the same room and dates; synchronization allows one request to succeed and prevents the other from creating a duplicate booking. The report also documents a fallback to in-memory mock storage when MySQL is unreachable.

7. Results and Output

The reported sample inventory contains five rooms: two Standard Single rooms at \$100 per night, two Deluxe Double rooms at \$175, and one Executive Suite at \$320. The sample analytics output records two reservation records, with one active confirmed booking and one completed booking. The interface contains sections for front-desk reservations, room search, billing and checkout, and reports or maintenance. Reservation records display the reservation ID, guest, room, dates, and status.

8. Output Screen

The following image is taken from the output/interface section of the supplied project report. It shows the documented Hotel Reservation & Management System screen, including the front-desk reservation form, active reservations table, and navigation area

```
+-----+
| Hotel Reservation & Management System | - [X] |
+-----+
| [ Front Desk & Reservations ] [ Room Search ] [ Billing & Checkout ] [ Reports & Maintenance ] |
+-----+
| +- Guest & Room Booking Details -+ |
| | Full Name: [ John Doe ] | Email Address: [ john@test.com ] |
| | Phone Number: [ 555-0199 ] | Room Number: [ 101 ] |
| | Check-In: [ 2026-09-10 ] | Check-Out: [ 2026-09-13 ] |
| | [ Confirm Booking ] | [ Cancel Selected Res ] |
+-----+
| +- Active Reservations -+ |
| | Res ID | Guest Name | Room # | Room Type | Check-In | Check-Out | Status |
| |-----|-----|-----|-----|-----|-----|-----|
| | 1001 | John Doe | 101 | Standard Single | 2026-09-10 | 2026-09-13 | CONFIRMED |
| | 1002 | Jane Smith | 201 | Deluxe Double | 2026-09-12 | 2026-09-15 | COMPLETED |
+-----+

(Note for submission: Run javac HotelSystemApp.java && java HotelSystemApp to bring up the
Swing window, perform a sample booking, and capture actual system screenshots for the UI, search
tab, and analytics report tab).
```

9. Sample Invoice and System Output

The supplied report documents a sample invoice for Reservation ID 1001. The reservation uses Room 101, with check-in on September 10, 2026 and check-out on September 13, 2026, giving a three-night stay. The reported total paid is \$336.00 and the status is PAID & CHECKED OUT. The analytics section also lists room inventory and active guest audit information.

10. Conclusion

The Hotel Reservation and Management System demonstrates the application of Java programming concepts to a practical hotel-management problem. Its modular design separates data objects, database access, business rules, and the graphical interface. The synchronization mechanism directly addresses concurrent booking conflicts, while billing, reporting, and serialized backup extend the system beyond basic reservation entry. The documented tests show that the application validates reservation input, rejects conflicting bookings, calculates charges, and supports recovery of stored reservation information.

Source basis: This report is an independently worded rewrite based on the supplied Hotel Reservation and Management System project report. Technical details and documented sample results have been retained from that source rather than copied verbatim.

cannot both confirm the same room and dates.

- * Input capture: collect guest name, email, phone, room number, check-in date, and check-out date.
- * Validate input: reject blank values and ensure the check-out date is later than the check-in date.
- * Acquire bookingLock before checking or modifying shared reservation data.
- * Verify that the requested room exists in the inventory.
- * Compare the requested dates with confirmed reservations for the same room.
- * Use the overlap rule: new check-in < existing check-out AND new check-out > existing check-in.
- * If an overlap is found, raise RoomUnavailableException and reject the booking.

5810 / 10,000 chars - 824 Words

RESERVATION

Fingerprint PRO ☐

Scan Mode

Normal Scan ☒

Deep Scan PRO ☐

Academic Scan PRO ☐

 Check Plagiarism via URL Exclude URLs

✓ PCI & NIST Compliant  

